

シェルスクリプト入門

Bash の読み方・書き方 ゼロから学ぶ

本書は Linux のシェルスクリプト（.sh
ファイル）を初めて読む方を対象に、基礎から実践的なパターンまでを解説します。

目次

第1章 シェルスクリプトとは

第2章 基本的な書き方

第3章 変数

第4章 引数

第5章 条件分岐 (if / case)

第6章 繰り返し (for / while)

第7章 関数

第8章 ファイル操作と入出力

第9章 よく使うコマンド

第10章 パイプとリダイレクト

第11章 プロセス制御

第12章 実践的なパターン集

第1章 シェルスクリプトとは

1.1 シェルとは

シェルとは、ユーザーがコンピュータに命令を与えるためのプログラムです。ターミナル（黒い画面）に文字を打ち込むと、それを受け取って実行するのがシェルです。

Linux で最もよく使われるシェルは bash（バッシュ）です。

1.2 シェルスクリプトとは

シェルへの命令を 1行ずつテキストファイルに書いたものが シェルスクリプト です。拡張子は .sh が慣習です。

複数のコマンドをまとめて自動実行できるため、繰り返し作業の自動化に使われます。

【シェルスクリプトの例】

```
#!/bin/bash
echo "こんにちは"
echo "今日は $(date) です"
```

1.3 実行方法

① ファイルに実行権限を付ける

```
chmod +x hello.sh
```

② 実行する

```
./hello.sh
```

または bash を明示して実行

```
bash hello.sh
```

☒ sudo をつけると管理者権限で実行できます（例: sudo bash setup.sh）

第2章 基本的な書き方

2.1 シバン行（1行目）

スクリプトの1行目には必ず シバン行 を書きます。どのシェルで実行するかを宣言します。

```
#!/bin/bash
```

☒ "#!" はシャープバンと読みます。/bin/bash は bash の場所です。

2.2 コメント

から行末までは コメント です。プログラムとして実行されません。

```
# これはコメントです（実行されない）
```

```
echo "hello" # 行末コメントも書けます
```

2.3 set -e（エラーで即停止）

スクリプトの先頭には書くと、いずれかのコマンドが失敗した瞬間にスクリプト全体を停止します。バグを早期に発見できるため、実用スクリプトでは必ず書きます。

```
#!/bin/bash
```

```
set -e # エラーで即停止
```

```
set -u # 未定義変数の参照でエラー（省略可）
```

```
set -o pipefail # パイプ内のエラーも検知（省略可）
```

2.4 echo（画面に表示）

```
echo "メッセージを表示"
```

```
echo "計算結果: $((3 + 5))" # 算術演算
```

```
echo "" # 空行
```

第3章 変数

3.1 変数の代入と参照

```
# 代入 (= の前後にスペースを入れてはいけない)
NAME="Alice"
AGE=25
DURATION=60
```

```
# 参照 ($ をつける)
echo "$NAME" # → Alice
echo "${NAME}" # 波カッコで囲むと確実 (推奨)
echo "${AGE}歳" # 文字列との連結
```

☑ 変数名は大文字が慣習です (小文字でも動きます)。

3.2 デフォルト値付き変数

変数が未設定のとき、デフォルト値を使いたい場合の書き方です。

```
# ${変数名:-デフォルト値}
DURATION="${1:-60}" # 第1引数が渡されなければ 60 を使う
TAG="${2:-test}" # 第2引数が渡されなければ "test" を使う

echo "Duration: $DURATION"
```

3.3 コマンドの出力を変数に格納

コマンドの実行結果を変数に代入したいとき、\$() で囲みます。

```
# $( ) の中のコマンドを実行し、出力を代入
TODAY=$(date +%Y%m%d)
HOSTNAME=$(hostname)
USER_LIST=$(cat /etc/passwd | cut -d: -f1)

echo "今日: $TODAY"
echo "ホスト名: $HOSTNAME"
```

3.4 数値の計算

```
A=10
B=3
```

```
echo $((A + B)) # → 13
echo $((A - B)) # → 7
echo $((A * B)) # → 30
echo $((A / B)) # → 3 （整数除算）
echo $((A % B)) # → 1 （余り）
```

3.5 特殊変数

変数	意味	例
\$0	スクリプト自身のファイル名	./setup.sh
\$1～\$9	引数（1番目～9番目）	60, normal
\$@	全引数（リスト）	60 normal tag
\$#	引数の数	3
\$?	直前のコマンドの終了コード（0=成功）	0
\$\$	現在のプロセスID	12345
\$_	直前のバックグラウンドプロセスID	12346

第4章 引数

4.1 引数の受け取り方

スクリプトを実行するときに値を渡せます。これを **引数** と呼びます。

```
# 実行例
bash measure.sh 60 normal 20260706

# スクリプト内
DURATION=$1 # → 60
SCENARIO=$2 # → normal
TAG=$3 # → 20260706
```

4.2 引数のチェック

```
#!/bin/bash
set -e

DURATION="${1:-60}" # 引数1 (なければ60)
SCENARIO="${2:-normal}"

# 引数が数値かチェック
if ! [[ "$DURATION" =~ ^[0-9]+$ ]]; then
    echo "エラー: durationは数値で指定してください"
    exit 1
fi

echo "計測時間: ${DURATION}秒"
echo "シナリオ: $SCENARIO"
```

☒ `exit 1` はスクリプトを異常終了させます (0 = 正常終了)。

第5章 条件分岐 (if / case)

5.1 if 文の基本

```
if [ 条件 ]; then
    # 条件が真のときの処理
elif [ 別の条件 ]; then
    # 別の条件が真のときの処理
else
    # どれも真でないときの処理
fi
```

5.2 文字列の比較

```
NAME="normal"

if [ "$NAME" = "normal" ]; then
    echo "normalです"
fi

if [ "$NAME" != "failure" ]; then
    echo "failureではありません"
fi

# 変数が空かどうか
if [ -z "$NAME" ]; then
    echo "NAMEは空です"
fi

if [ -n "$NAME" ]; then
    echo "NAMEに値があります"
fi
```

5.3 数値の比較

書き方	意味
["\$A" -eq "\$B"]	A = B (等しい)
["\$A" -ne "\$B"]	A ≠ B (等しくない)
["\$A" -lt "\$B"]	A < B (小さい)
["\$A" -le "\$B"]	A ≤ B (以下)

<code>["\$A" -gt "\$B"]</code>	A > B （大きい）
<code>["\$A" -ge "\$B"]</code>	A ≥ B （以上）

```
A=10; B=5
```

```
if [ "$A" -gt "$B" ]; then
    echo "Aの方が大きい"
fi
```

5.4 ファイル・ディレクトリの存在チェック

```
FILE="/tmp/result.csv"
DIR="/tmp/results"

if [ -f "$FILE" ]; then # ファイルが存在する
    echo "ファイルあり"
fi

if [ -d "$DIR" ]; then # ディレクトリが存在する
    echo "ディレクトリあり"
fi

if [ ! -f "$FILE" ]; then # ! で否定
    echo "ファイルなし"
fi
```

5.5 正規表現マッチ（[[]] を使う）

より強力な条件チェックには [[]] を使います。

```
VALUE="60"

# 数値かどうか
if [[ "$VALUE" =~ ^[0-9]+$ ]]; then
    echo "数値です"
fi

# "normal" か "failure" か
SCENARIO="normal"
if [[ "$SCENARIO" =~ ^(normal|failure|failure_reroute)$ ]]; then
    echo "有効なシナリオ"
fi
```

5.6 case 文（複数の値の分岐）

if/elif が多くなるときは case 文の方がスッキリ書けます。

```
SCENARIO="failure"

case "$SCENARIO" in
normal)
    echo "通常シナリオの処理"
    ;;
failure)
    echo "障害シナリオの処理"
    ;;
failure_reroute)
    echo "迂回シナリオの処理"
    ;;
*) # どれにも一致しない場合
    echo "不明なシナリオ: $SCENARIO"
    exit 1
    ;;
esac
```

☒ 各ケースは ;; で終わります（忘れると次のケースに「落ちて」しまいます）。

第6章 繰り返し (for / while)

6.1 for ループ (リストを順番に処理)

```
# 値のリストをループ
for SCENARIO in normal failure failure_reroute; do
    echo "処理中: $SCENARIO"
done

# 数値の範囲
for i in 1 2 3 4 5; do
    echo "番号: $i"
done

# seq を使った数値ループ
for i in $(seq 1 10); do
    echo "カウント: $i"
done
```

6.2 配列のループ

```
# 配列の定義
SCENARIOS=(normal failure failure_reroute)

# 配列のループ
for s in "${SCENARIOS[@]}"; do
    echo "シナリオ: $s"
done

# インデックス付きループ
for i in "${!SCENARIOS[@]}"; do
    echo "$i 番目: ${SCENARIOS[$i]}"
done
```

6.3 while ループ

```
# 条件が真の間繰り返す
COUNT=0
while [ "$COUNT" -lt 5 ]; do
    echo "カウント: $COUNT"
    COUNT=$((COUNT + 1))
done
```

```
done

# 無限ループ (Ctrl+C で止める)
while true; do
    echo "1秒ごとに実行"
    sleep 1
done
```

6.4 break と continue

```
for i in 1 2 3 4 5; do
    if [ "$i" -eq 3 ]; then
        continue # 3 をスキップして次へ
    fi
    if [ "$i" -eq 5 ]; then
        break # 5 でループを抜ける
    fi
    echo "$i"
done
# 出力: 1 2 4
```

第7章 関数

7.1 関数の定義と呼び出し

同じ処理を複数回使いたいときは 関数 にまとめます。

```
# 関数の定義（使う前に定義する必要がある）
greet() {
    echo "こんにちは、$1 さん" # $1 は関数への引数
}

# 呼び出し
greet "Alice" # → こんにちは、Aliceさん
greet "Bob" # → こんにちは、Bobさん
```

7.2 local 変数

関数の中だけで使う変数は local をつけます。

local をつけないと、スクリプト全体に影響する「グローバル変数」になります。

```
calculate() {
    local A=$1 # local: この関数内だけで有効
    local B=$2
    local RESULT=$((A + B))
    echo "$RESULT"
}

SUM=$(calculate 10 20)
echo "合計: $SUM" # → 合計: 30
```

7.3 戻り値

bash の関数は数値（0～255）しか return できません。文字列を返したいときは echo を使います。

```
# 成功/失敗を返す
check_file() {
    local FILE=$1
    if [ -f "$FILE" ]; then
        return 0 # 成功
    else
        return 1 # 失敗
    fi
}
```

```
if check_file "/tmp/result.csv"; then
    echo "ファイルあり"
fi

# 文字列を返す (echo + コマンド置換)
get_date() {
    echo "$(date +%Y%m%d)"
}

TODAY=$(get_date)
echo "今日: $TODAY"
```

第8章 ファイル操作と入出力

8.1 基本的なファイル操作

```
# ファイルの作成・書き込み
echo "内容" > file.txt # 新規作成（上書き）
echo "追加" >> file.txt # 末尾に追記

# ファイルの読み込み
cat file.txt # 全内容を表示

# ファイル・ディレクトリの操作
mkdir -p /tmp/results # ディレクトリ作成（-p: 親ディレクトリも作る）
cp src.txt dst.txt # コピー
mv old.txt new.txt # 移動・名前変更
rm file.txt # 削除
rm -f file.txt # 強制削除（なくてもエラーにしない）
rm -rf /tmp/old_dir # ディレクトリごと削除（注意！）
```

8.2 リダイレクト（出力の向き先を変える）

```
# 標準出力（通常の出力）を ファイルに保存
ls -la > list.txt # 上書き
ls -la >> list.txt # 追記

# 標準エラー出力を捨てる（エラーメッセージを表示しない）
command 2>/dev/null

# 標準出力とエラー出力の両方を捨てる
command > /dev/null 2>&1

# 標準出力をファイルに、エラーは画面に
command > output.txt

# 標準入力をファイルから読む
command < input.txt
```

8.3 CSV ファイルへの書き込み（実験でよく使うパターン）

```
CSV="/tmp/results.csv"
```

```
# ヘッダー行を書く
echo "time,value1,value2" > "$CSV"

# データを追記
for i in 1 2 3; do
    echo "$i,$((i*10)),$((i*20))" >> "$CSV"
done

# 中身を確認
cat "$CSV"
# time,value1,value2
# 1,10,20
# 2,20,40
# 3,30,60
```

第9章 よく使うコマンド

9.1 ファイル検索・テキスト処理

```
# grep: パターンにマッチする行を抽出
grep "error" logfile.txt # "error" を含む行
grep -v "error" logfile.txt # "error" を含まない行
grep -i "error" logfile.txt # 大文字小文字を無視

# cut: 列を抽出
cut -d, -f1,3 data.csv # カンマ区切りの1列目と3列目

# sort: 並び替え
sort data.txt # 昇順
sort -r data.txt # 降順
sort -n numbers.txt # 数値として並び替え
sort -k2 -t, data.csv # カンマ区切りの2列目で並び替え

# head / tail: 先頭・末尾
head -10 file.txt # 最初の10行
tail -10 file.txt # 最後の10行
tail -f logfile.txt # リアルタイムで追跡
```

9.2 awk（列処理の強力なツール）

awk はテキストを列で処理するのが得意なコマンドです。

```
# 基本: $1=1列目, $2=2列目, $NF=最後の列
echo "Alice 25 Tokyo" | awk '{print $1}' # → Alice
echo "Alice 25 Tokyo" | awk '{print $1, $3}' # → Alice Tokyo

# -F で区切り文字を指定
awk -F, '{print $1, $2}' data.csv # カンマ区切り

# 条件付き抽出（2列目が100より大きい行）
awk -F, '$2 > 100 {print $0}' data.csv

# 合計を計算
awk -F, '{sum += $2} END {print "合計:", sum}' data.csv

# 特定の列の平均
awk -F, 'NR>1 {sum+=$2; n++} END {print sum/n}' data.csv
```

```
# ↑ NR>1 でヘッダー行をスキップ
```

9.3 sleep（待機）

```
sleep 1 # 1秒待つ  
sleep 0.5 # 0.5秒待つ  
sleep 10 # 10秒待つ
```

9.4 date（日時）

```
date # 現在日時  
date +%Y%m%d # 20260706 形式  
date +%H:%M:%S # 12:34:56 形式  
date +%s # Unix タイムスタンプ（秒）  
date +%s%3N # ミリ秒タイムスタンプ
```

第10章 パイプとリダイレクト

10.1 パイプ (|)

パイプを使うと、あるコマンドの 出力を次のコマンドの入力 に渡せます。

```
# ls の出力を grep に渡してフィルタリング
ls -la | grep ".sh"

# 複数つなげる
cat logfile.txt | grep "error" | sort | uniq | head -5
# ↑エラー行だけ   ↑ソート   ↑重複除去   ↑先頭5行

# パイプで行数を数える
ls *.sh | wc -l # .sh ファイルの数
grep "DOWN" log.txt | wc -l # DOWN を含む行数
```

10.2 よく使うパイプの組み合わせ

```
# CSV の特定列の最大値
awk -F, '{print $2}' data.csv | sort -n | tail -1

# 特定プロセスの PID を取得
pgrep -f "iperf3" | head -1

# ログから最新のエラーを抽出
tail -100 logfile.txt | grep "ERROR"

# コマンドの出力に行番号を付ける
cat file.txt | nl
```

10.3 && と || （条件付き実行）

```
# && : 前のコマンドが成功したときだけ次を実行
mkdir /tmp/results && echo "ディレクトリ作成成功"
command1 && command2 # command1 が成功なら command2 を実行

# || : 前のコマンドが失敗したときに次を実行
command || echo "失敗しました"
command || exit 1 # 失敗したら終了

# よく使うパターン
```

```
command 2>/dev/null || true # エラーを無視して続ける  
[ -d "$DIR" ] || mkdir -p "$DIR" # なければ作る
```

第11章 プロセス制御

11.1 バックグラウンド実行 (&)

コマンドの末尾に & をつけると、バックグラウンドで実行されます。その間も次のコマンドを続けられます。

```
# バックグラウンドで実行
long_command &
PID=$! # バックグラウンドプロセスの ID を保存
echo "PID: $PID"

# 別の処理を続ける
echo "処理中..."
sleep 5

# バックグラウンドの終了を待つ
wait $PID
echo "完了"
```

11.2 プロセスの終了

```
PID=12345

kill "$PID" # 終了シグナルを送る（プロセスに後処理の機会を与える）
kill -9 "$PID" # 強制終了（即座に停止）
kill -0 "$PID" # プロセスが生きているか確認（終了コードで判断）

# プロセスが生きているか確認
if kill -0 "$PID" 2>/dev/null; then
    echo "実行中"
else
    echo "終了済み"
fi
```

11.3 pkill / pgrep（名前でプロセス操作）

```
# 名前でプロセスを検索
pgrep -f "iperf3" # iperf3 プロセスの PID 一覧
pgrep -f "iperf3" | head -1 # 最初の1つ

# 名前でプロセスを終了
```

```
pkill -f "iperf3" # iperf3 を全て終了  
pkill -9 -f "iperf3" # iperf3 を強制終了
```

11.4 複数のバックグラウンドプロセスを管理

```
# 複数を並列実行  
proc1 &  
PID1=$!  
proc2 &  
PID2=$!  
proc3 &  
PID3=$!  
  
# 全部の終了を待つ  
wait $PID1 $PID2 $PID3  
echo "全プロセス完了"  
  
# クリーンアップ（スクリプト終了時に確実に停止）  
cleanup() {  
    kill "$PID1" 2>/dev/null || true  
    kill "$PID2" 2>/dev/null || true  
}  
trap cleanup EXIT # EXIT シグナルで cleanup を呼ぶ
```

第12章 実践的なパターン集

12.1 Docker コンテナへのコマンド実行

このプロジェクトでよく使うパターンです。

```
# コンテナ内でコマンドを実行
docker exec コンテナ名 コマンド

# 例
docker exec LER_Ingress ip route show
docker exec CR1 tc qdisc show dev cr1-lere

# バックグラウンドで実行 (-d)
docker exec -d Tx1 iperf3 -c 10.20.1.1 -u -b 2G -t 60

# 複数コマンドを実行
docker exec LER_Ingress bash -c "ip route flush table 41 && ip route add ..."
```

12.2 SSH でリモートコマンド実行

```
SSH_OPT="-i ~/.ssh/lab_key -o StrictHostKeyChecking=no"
SW1="kannolab@192.168.128.33"

# リモートでコマンド実行
ssh $SSH_OPT "$SW1" "ls /home/kannolab/scripts"

# リモートでスクリプト実行
ssh $SSH_OPT "$SW1" "sudo bash /home/kannolab/scripts/setup.sh"

# SCP でファイル転送
scp $SSH_OPT ./script.sh "$SW1":/home/kannolab/scripts/
scp $SSH_OPT "$SW1":/tmp/result.csv ./results/
```

12.3 ループ内でのエラー処理

```
for HOST in sw1 sw2 sw3; do
    echo "=== $HOST の処理 ==="
    ssh "$HOST" "sudo bash setup.sh" && echo " [OK] $HOST" || echo " [WARN] $HOST
失敗（継続）"
done
```

12.4 タイムスタンプを使ったログ出力

```
log() {
    echo "[$(date +%H:%M:%S)] $*"
}

log "処理開始"
sleep 2
log "処理完了"
# 出力:
# [12:34:56] 処理開始
# [12:34:58] 処理完了
```

12.5 スクリプトのディレクトリを取得

```
# このスクリプト自体がある場所を取得（絶対パスで）
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

# スクリプトから見た相対パスで他のファイルを参照
source "${SCRIPT_DIR}/lab_config.sh"
bash "${SCRIPT_DIR}/frr_dscp_te.sh"
```

☒ `$BASH_SOURCE[0]` はスクリプト自身のパスです。source で読み込まれた場合でも正しく動きます。

12.6 数値の単位変換（G/M/K → 数値）

このプロジェクトで使っているパターンです。

```
# "10G" → 10000, "30M" → 30, "500K" → 0.5 に変換
to_mbps() {
    local r
    r=$(echo "$1" | tr '[:upper:]' '[:lower:]') # 小文字に変換
    case "$r" in
        *g) echo $(( ${r%g} * 1000 )) ;; # G → ×1000
        *m) echo "${r%m}" ;; # M → そのまま
        *k) echo "0" ;; # K → 0 (Mbps換算で無視)
        *) echo "${r//[0-9]/}" ;; # 数字だけ抽出
    esac
}

RATE="10G"
echo "$(to_mbps "$RATE") Mbps" # → 10000 Mbps
```

12.7 スクリプトの読み方ガイド（まとめ）

未知のスクリプトを読むときの手順：

1. `#!/bin/bash` と `set -e` を確認（どう動くかの基本設定）
2. 変数定義の塊を読む（何が設定されているか把握）
3. 関数定義を読む（実際の処理のまとめ）
4. 末尾のメイン処理を読む（全体の流れ）
5. わからないコマンドは：
 `man コマンド名` `#` マニュアルを読む
 `コマンド --help` `#` ヘルプを表示

以上